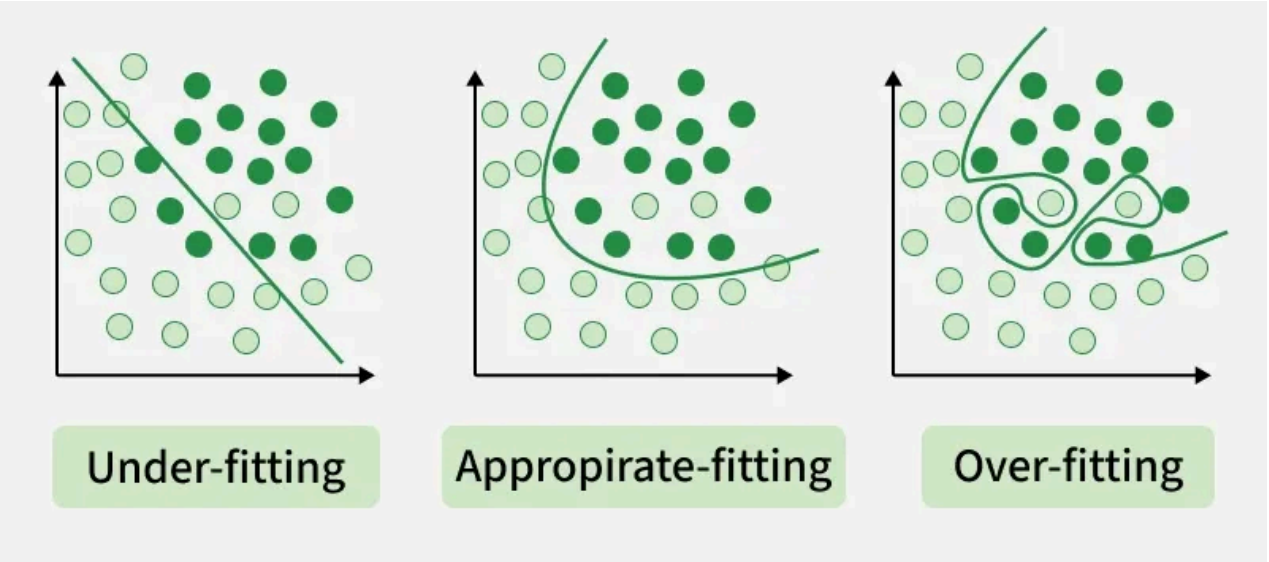
Regularization

it plays an important role in in high-dimensional problem.
The test error tend to de0crease as the number of features increase

- **Prevent overfitting**: add constraint to the models to reduce the risk of memorizing noise in the training data
- **Improves generalization**: encourages simpler models that performs better on new unseen data



There are 3 type of regularization

1. Lasso Regression (L1 Regularization)

Lasso = Least Absolute Shrinkage and Selection Operator
It add the **absolute value to the magnitude** of the coefficient as a **penalty term** to the **loss function** which can **shrink some coefficients to zero** which helps in selecting only the important features and ignoring the less important one.

-selecting only best feature

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} |w_i|$$

Where

- m: Number of Features
- n: Number of Examples
- y_i: Actual Target Value
- $\hat{y}_i$: Predicted Target Value

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")

print("Coefficients:", lasso.coef_)
```

2. Ridge Regression (L2 Regularization)

It adds the **squared magnitude** of the coefficient as **a penalty term to the loss function** which handles multicollinearity by **shrinkage the coefficients of correlated features instead of eliminating them.**
-help with multicollinearity

This force the Learning algorithm to not only fit the data but also keep the model weights as small as possible.
Note that the regularization term. should only be added to the cost function during the training. Once the model is trained, you want to evaluate the model's performance using the unregularized performance measures.

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} w_i^2$$

Where,

- n: Number of examples or data points
- m: Number of features i.e predictor variables
- y_i: Actual target value for the ith example
- $\hat{y}_i$: Predicted target value for the ith example
- w_i: Coefficients of the features
- λ: Regularization parameter that controls the strength of regularization

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
print("Coefficients:", ridge.coef_)
```

3. Elastic Net Regression

It is **a combination of L1 and L2 regularization**, that we **add the absolute norm of the weights as well as the squared measure of the weights**. With the help of an extra **hyperparameter** that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\left((1-\alpha)\sum_{i=1}^{m}|w_i| + \alpha\sum_{i=1}^{m} w_i^2\right)$$

Where

- n: Number of examples (data points)
- m: Number of features (predictor variables)
- y_i: Actual target value for the ith example
- $\hat{y}_i$: Predicted target value for the ith example
- wi: Coefficients of the features
- λ: Regularization parameter that controls the strength of regularization
- α: Mixing parameter where $0 \leq \alpha \leq 1$ and α= 1 corresponds to Lasso (L_1) regularization, α= 0 corresponds to Ridge (L_2) regularization and Values between 0 and 1 provide a balance of both L1 and L2 regularization

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = ElasticNet(alpha=1.0, l1_ratio=0.5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Coefficients:", model.coef_)
```

Benefits of Regularization

- **Prevents Overfitting:** Regularization helps models focus on underlying patterns instead of memorizing noise in the training data.
- **Enhances Performance:** Prevents excessive weighting of outliers or irrelevant features helps in improving overall model accuracy
- **Stabilizes Models:** Reduces sensitivity to minor data changes which ensures consistency across different data subsets.
- **Prevents Complexity:** Keeps model from becoming too complex which is important for limited or noisy data.

- **Handles Multicollinearity:** Reduces the magnitudes of correlated coefficients helps in improving model stability.
- **Promotes Consistency:** Ensures reliable performance across different datasets which reduces the risk of large performance shifts.

When and What to use

Case 1: All predictors matter

- Ridge better
- Because shrinking all slightly is good
  Case 2: Few predictors matter
- Lasso better
- Because it removes noise variables

Geek4Geek and ISL.py